Cadence

Technical Architecture & Build Plan

v1.0 — Companion to the Product Specification

Prepared for: Tripp McInnis

Author: Claude (with Tripp)

Date: May 2026

Companion doc: Cadence_Product_Spec.docx

1. Approach

This document is the build playbook. It maps every product requirement from the spec to a concrete technical decision, sequences the work into phases you can hand to Claude Code week by week, and surfaces the risks you should know about before writing the first line of Swift.

Guiding principles

- **Boring tech where possible.** SwiftUI + SwiftData + CloudKit is Apple's blessed path. It is free, robust, and lets you build for iPhone and Mac from one codebase. No Firebase, no custom backend, no Electron.
- **Local-first.** Every interaction must work fully offline. CloudKit syncs in the background; the app is never blocked on a network call.
- **One model, every surface.** The data layer is the same whether the consumer is the iPhone UI, a widget, Siri, or Claude. No bespoke APIs per surface.
- **Ship-then-improve.** MVP is functional and beautiful but lean. AI and exotic features come in v2 once daily use confirms the foundation.
- **Hand-off ready.** Code is structured so Claude Code (or a future contractor) can take any phase end-to-end with the spec + this doc as context.

2. Stack & Frameworks

Every choice below is the default Apple path unless noted. Solo-builder discipline: avoid anything that pulls you into vendor lock-in or off-platform complexity until v3.

Layer	Choice	Why
UI framework	SwiftUI	Single codebase iPhone + iPad + Mac. Built-in dynamic type, accessibility, animations.
Min target	iOS 17, macOS 14	Lets us use interactive widgets, observable, App Intents 2.
Language	Swift 5.9+	Async/await, observable macros.
Local store	SwiftData	Codable-first, simpler than Core Data, plays nicely with CloudKit.
Sync (private)	CloudKit Private DB	Native iCloud, free, end-to-end encrypted, zero server.
Sync (shared)	CloudKit Shared DB (CKShare)	Per-list sharing model maps cleanly to CKShare.
Push	APNs via CloudKit subscriptions	Silent push to wake clients for instant shared-list updates.
Calendar	Google Calendar API + AppAuth-iOS (OAuth)	Read-only events. EventKit (Apple Calendar) added v1.1.
Notifications	UserNotifications + UNTimeIntervalNotificationTrigger	Local for task reminders; CloudKit for shared changes.
Widgets	WidgetKit + AppIntents	Interactive widgets on iOS 17. Lock Screen accessory family.
Siri (v2)	AppIntents framework	Modern Siri Shortcuts. No Intents Extension needed.
AI (v2)	Anthropic API (claude-sonnet-4-6)	Direct HTTPS calls. App-key (subscription) or BYOK.
Crash/analytics	MetricKit + TelemetryDeck	Privacy-preserving, no Firebase. Optional opt-in.
Build/CI	Xcode Cloud	Native, zero-config, free for personal use.
Distribution	TestFlight → App	TestFlight forever for personal use; App

Layer	Choice	Why
	Store	Store launch optional.

On not picking React Native / Flutter

React Native gets you Android for free, but the cost is real on iPhone: clunky widgets, second-class Siri, no proper Lock Screen accessories, weak interactive widgets, and a constant battle with iOS conventions. Cadence's value is being a beautiful iPhone-first app. If the co-owner ends up needing Android, we add a thin web companion in v1.1 instead of rebuilding the whole stack.

On not picking Firebase / Supabase

CloudKit gives us iCloud sign-in (zero friction for you), end-to-end encryption, sharing primitives (CKShare), and Apple's free tier that easily covers personal-app scale. The downside — no Android client — is solved by the same web-companion plan above if needed.

3. Data Model

Everything is a SwiftData model with @Model. Below is the schema in plain Swift, simplified for readability.

3.1 Task

```
@Model final class Task {
 @Attribute(.unique) var id: UUID
 var title: String
 var notes: String?
 var dueDate: Date?            // date+time when present
 var allDay: Bool              // ignore time portion when true
 var priority: Priority        // none | low | medium | high
 var status: Status            // open | completed | snoozed
 var completedAt: Date?
 var snoozeUntil: Date?
 var tags: [String]            // free-form
 var reminderOffsets: [TimeInterval]  // 0 = at due, -1800 = 30 min before
 var createdAt: Date
 var createdBy: String         // CKRecord user ID
 var completedBy: String?
 // Relationships
 var list: TaskList?           // belongs to one list
 var parent: Task?             // nil for top-level
 @Relationship(.cascade) var subtasks: [Task] = []
 // Computed
 var isCarriedOver: Bool {
  guard let due = dueDate else { return false }
  return status == .open && Calendar.current.startOfDay(for: due) < .now.startOfDay
 }
}
```

3.2 TaskList

```
@Model final class TaskList {
```

```
  @Attribute(.unique) var id: UUID
  var name: String
  var colorKey: String          // palette key, not raw hex
  var iconKey: String           // SF Symbol name
  var sortOrder: Int
  var rolloverPolicy: RolloverPolicy   // on | ask | off
  var defaultReminderOffsets: [TimeInterval]
  var isHidden: Bool
  // Sharing
  var shareRecordName: String?   // set when shared via CKShare
  @Relationship(.cascade) var tasks: [Task] = []
}
```

3.3 CachedEvent (Google Calendar)

Calendar events are NOT in CloudKit — they are cached locally and refreshed from Google.

```
@Model final class CachedEvent {
  @Attribute(.unique) var id: String     // Google event ID
  var calendarId: String
  var title: String
  var start: Date
  var end: Date
  var isAllDay: Bool
  var location: String?
  var attendees: [String]
  var meetingURL: URL?
  var lastFetched: Date
}
```

3.4 Connected accounts

```
@Model final class ConnectedAccount {
  @Attribute(.unique) var id: UUID
  var provider: Provider         // .google | .apple (v1.1)
  var email: String
```

```
  // Tokens stored in Keychain (NOT here)
  var calendars: [CalendarConfig]   // user-selected, with color overrides
}
```

3.5 Activity log (shared lists only)

```
@Model final class Activity {
  @Attribute(.unique) var id: UUID
  var list: TaskList?
  var taskId: UUID
  var actor: String              // CKRecord user ID
  var kind: ActivityKind         // .created | .completed | .edited | .deleted
  var at: Date
}
```

4. Sync Architecture

4.1 Topology

- Each user's private data lives in the iCloud Private Database for their Apple ID.
- Each shared list is materialized as a CKRecord zone and a CKShare. The owner creates it; collaborators accept the share via deep link or in-app invite.
- All clients subscribe to CKDatabaseSubscriptions for both private and shared databases — silent push wakes the app to refresh.
- Apple Calendar data is never in CloudKit — only the SwiftData mirror is.

4.2 Conflict resolution

- **Default: last-write-wins per field.** CloudKit tracks per-field timestamps; we merge in our app delegate when a CKRecord modifier returns a conflict.
- **Title edits within 5 seconds of each other surface as a non-blocking sheet.** User picks which version to keep.
- **Status flips (open ↔ completed) are eventually consistent.** Completing in two places at once just lands on "completed," no conflict.

4.3 Sharing flow (CKShare)

- Owner taps Share on a list → CKShare URL generated → presented in iOS share sheet → recipient gets a Cadence deep link.
- Recipient must have Cadence installed and signed into iCloud. (For non-Apple users, see web companion in v1.1.)
- Owner can change roles or revoke access via Settings → Sharing → list name.
- CloudKit handles invite, accept, and revoke flows natively — we just call the APIs.

4.4 Background refresh

- Silent push (CKDatabaseSubscription) wakes the app for incoming changes — both private and shared.
- BGProcessingTask scheduled daily for Google Calendar refresh (15-min cadence when foreground, 1 hr in background).
- Widget timeline reloads on task add, complete, or sync — via WidgetCenter.shared.reloadAllTimelines().

5. Rollover Implementation

Rollover is the product's most important behavior. The implementation is dead simple — it's a computed property, not a migration.

5.1 How it works

- A task is "carried over" if dueDate < today's startOfDay AND status == .open.
- The Today view query is a SwiftData FetchDescriptor with a predicate that matches: (dueDate < tomorrow AND status == .open) OR (completedAt > yesterday AND status == .completed).
- Carried-over tasks are grouped at the top of the Today view by the SwiftUI layout, NOT by mutating the underlying date.
- Result: the task's actual dueDate is preserved (for history/analytics), and there is no data migration ever.

5.2 Per-task / per-list override

- Per-list rolloverPolicy controls whether tasks from this list show in the carried-over section, show as overdue (red chip), or stay hidden until the user opens that list.
- The "Ask each morning" mode triggers a sheet on first app open of the day; the sheet pulls the current carried-over set and offers Reschedule / Snooze / Bring forward.

5.3 Edge cases

- Snoozed tasks have status == .snoozed and a snoozeUntil date. They auto-flip to .open when snoozeUntil is reached, at which point rollover logic applies normally.
- Recurring tasks (v1.1) handle their own rollover via the RRULE engine.
- Long-gap (vacation) detection: if carried-over count > 10 on app open, show a triage sheet.

6. Google Calendar Integration

6.1 Auth

- Use AppAuth-iOS for OAuth 2.0 (Google's recommended SDK). ASWebAuthenticationSession in-app browser.
- Required scopes: https://www.googleapis.com/auth/calendar.readonly (MVP). Add events scope in v3 for two-way.
- Tokens stored in Keychain with kSecAttrAccessibleAfterFirstUnlock.
- Refresh token used silently. If revoked, surface reconnect banner in Today header.

6.2 Fetching

- Query: calendars.list once on connect; events.list per selected calendar with a sliding window timeMin/timeMax.
- Window: today − 7 days to today + 30 days. Refresh window roll-over happens on app foreground.
- Use ETag + If-None-Match for incremental pulls — cheap, cache-friendly.
- Store results in CachedEvent SwiftData entities; never put them in CloudKit.

6.3 Display rules

- All-day events appear in the date header strip in Today; timed events interleave with tasks by start time.
- Events are read-only in MVP. Tap → sheet with attendees + Open-in-Google-Calendar link.
- Multi-calendar overlays use the user's per-calendar color override.

6.4 Privacy

- Calendar payloads never leave the device.
- Anthropic API (v2) only receives event titles, not bodies or attendee details, unless the user opts in per request.

7. Notifications

7.1 Local notifications (per-task)

- UNUserNotificationCenter, scheduled with UNCalendarNotificationTrigger for each reminder offset.
- On task edit, we cancel previous identifiers and reschedule.
- Per-list default offsets stored on TaskList.defaultReminderOffsets.

7.2 Morning brief

- Single daily UNTimeIntervalNotificationTrigger fired at user-set time (default 7:30).
- Brief content computed at delivery time via UNNotificationServiceExtension reading SwiftData.

7.3 Shared-list pushes

- CloudKit subscriptions deliver silent push when a shared CKRecord changes.
- App processes change → if it matches user's notification preferences (everything / @mentions / off), schedules a local notification.

7.4 Implementation gotchas

- Stay under the 64 pending notifications limit — coalesce reminders when over.
- Time-sensitive interruption level used for at-due-time reminders; passive for daily brief.
- On macOS, use the same UNUserNotificationCenter API — they unify in Sonoma+.

8. Widgets & Lock Screen

8.1 Architecture

- WidgetKit + AppIntents. The widget extension reads SwiftData via an App Group (group.net.mcinnis.cadence).
- App Group enables shared container between main app, widget, and Siri extensions.
- All writes go through the main app; widgets are read-only consumers.

8.2 Sizes & families

- Home Screen: systemSmall, systemMedium, systemLarge.
- Lock Screen: accessoryCircular, accessoryRectangular, accessoryInline (iOS 16+).
- StandBy on iPhone (iOS 17+): same accessoryRectangular auto-scales.

8.3 Interactive widgets (iOS 17+)

- A tap on a task row triggers a custom AppIntent (CompleteTaskIntent) that completes the task in the shared container.
- WidgetCenter.shared.reloadAllTimelines() called after every relevant write.

8.4 Refresh policy

- TimelineProvider returns entries every 15 minutes (system minimum) plus on-demand reloads.
- On task complete / add / move, widgets are reloaded immediately.

9. Siri Shortcuts (v2)

9.1 App Intents (modern)

Use the AppIntents framework introduced in iOS 16 — no Intents Extension or .intentdefinition files. Cleaner, type-safe Swift.

9.2 Intents we ship

- **AddTaskIntent.** Parameters: title, list, dueDate. Siri dialog disambiguates list if multiple match.
- **WhatsOnMyPlateIntent.** Reads Today view summary aloud and shows in Siri UI.
- **CompleteTaskIntent.** Fuzzy-matches against open tasks in Today; confirms before completing if confidence < 0.85.
- **OpenCadenceIntent.** Deep link to Today view; used in Shortcuts/Personal Automations like "every morning at 7am."

9.3 Donation

- Donate AddTaskIntent and OpenCadenceIntent regularly so Siri Suggestions surface them.
- Custom phrases registered in App Shortcut Provider.

10. Claude Integration (v2)

10.1 Where Claude lives

- Three product surfaces: Plan my day, Natural-language capture (long-press +), and Weekly review.
- All call the Anthropic Messages API (claude-sonnet-4-6) directly over HTTPS from the device.
- No middleware server in MVP — direct from device. Move to a Cloudflare Worker proxy only if (a) we ship with a managed key under a subscription model, or (b) prompt versioning becomes painful.

10.2 Plan my day

Input

- Today's open tasks (title, list, priority, dueDate).
- Calendar events for today (start, end, title).
- User-set work hours and break preferences.
- Optional: yesterday's completion patterns (carried-over count, finish times).

Prompt structure

```
System: You are Cadence's day-planning assistant. Output STRICT JSON only.
User: Plan today. Tasks: [...]. Events: [...]. Work hours: 8:30–18:00.
      Free blocks: [10:00–11:00, 14:00–17:00].
Tool: ReturnPlan(blocks: [{ start, end, taskId|note, kind: task|event|break }], commentary: string)
```

Response handling

- Validate JSON against a Swift Codable schema. Reject and retry on malformed output.
- Render blocks as draft cards in the Plan sheet.
- On Apply: write block times to tasks (no Calendar mutation until v3 two-way sync).

10.3 Natural-language capture

Flow

- Long-press +. Voice input via SFSpeechRecognizer (local).
- Transcribed text → Claude prompt asking for structured task(s): title, dueDate, list, priority, tags, subtasks.
- Render confirm/edit sheet. User taps Save to commit.

Why not local NL parsing

MVP does deterministic NL parsing for simple cases (date, time, #list, !priority) using regex/NSDataDetector — that ships in v1.0. Claude adds the flexible cases ("every other Tuesday until summer ends", "call Mom before her birthday") in v2.

10.4 Weekly review

- Triggered Friday afternoon via local notification + Today header banner.
- Sends summary of completed tasks (titles), carried-over patterns (counts by day), and calendar density.
- Receives: a written review (under 200 words) + 3 suggested focus areas for next week.

10.5 Cost & key management

- **Default: app-managed key.** If shipped publicly, gated behind Cadence Plus ($3.99/mo or $29.99/yr). Per-user budget cap default $0.20/day; soft warning then hard cap.
- **Power-user: BYOK.** Settings → AI → Use my own Anthropic key. Stored in Keychain.
- **Cost estimate.** Plan my day call: ~1500 input + 800 output tokens = roughly $0.012 per call with Claude Sonnet 4.6. NL capture: ~$0.003 per task. Weekly review: ~$0.02. A power user running everything costs ~$1.50/month.

10.6 Privacy

- Task titles + event titles only sent by default.
- Task notes opt-in per request (toggle in the sheet).
- All exchanges logged to Settings → AI History; user can delete or export.
- Anthropic API requests include no PII other than what the user typed.
- API key never leaves Keychain; never logged.

11. Build Plan

Phased plan you can execute with Claude Code, one phase per week or two. Each phase ends in a testable state — TestFlight build, real device, dogfood.

Phase	Duration	Deliverable	Done when...
Phase 0	~1 week	Project setup, app icon, asset pipeline, design tokens, CI	Xcode project compiles to TestFlight.
Phase 1	~2 weeks	Data model + local CRUD (no sync yet). Today view, Lists view, Task detail. Rollover logic.	You can add, edit, complete, and roll over tasks on one device.
Phase 2	~1.5 weeks	CloudKit private sync between iPhone + Mac (if Mac target enabled now) or two iPhones for testing.	Two devices stay in sync within 2s on Wi-Fi.
Phase 3	~1 week	Google Calendar OAuth + read-only events display, interleaved in Today and Week views.	Your real Google events show up correctly.
Phase 4	~1 week	CloudKit sharing (CKShare). Joint Business list shared with co-owner. Activity attribution.	Co-owner can edit Joint Business and you see changes within seconds.
Phase 5	~1 week	Notifications: per-task reminders, morning brief, daily summary.	Reminders fire at the right time without duplicates.
Phase 6	~1.5 weeks	Widgets — small/medium/large + Lock Screen accessories.	Today widget shows real data and updates on task changes.
Phase 7	~1 week	Polish pass: animations, haptics, empty states, onboarding, App Store screenshots.	You hand it to a friend who finds nothing rough.
MVP Ship		v1.0 on TestFlight (or App Store).	Total: ~8–9 weeks of focused build.
v1.1	~3 weeks	Mac app refinement, recurring tasks, smart filters, Watch complication.	Mac is daily-usable; recurring tasks work.
v2	~3–4 weeks	Siri Shortcuts (App Intents) + Claude integration (Plan my day, NL capture, weekly review).	"Plan my day" produces a usable plan and Siri can add tasks hands-free.

Working with Claude Code

- Each phase becomes a focused Claude Code session. Give it the spec, this doc, and the phase scope.
- Commits are the contract. Every phase ends with green tests + a TestFlight build.
- Use a personal GitHub repo (private). Xcode Cloud builds on push.
- Keep design tokens (colors, type, spacing) in a single Tokens.swift file — Claude Code can adjust the entire app from there.

12. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
CloudKit sharing edge cases (offline merges, role changes)	Medium	High	Build a small test harness with two simulators; document conflict scenarios; fall back to last-write-wins with conflict surfacing UI.
Co-owner is on Android	Unknown	High	Confirm device early in Phase 0. If Android: scope a lightweight web app in v1.1 instead of Mac-first, with Firebase or Supabase as cross-platform sync.
Google Calendar API quota / OAuth scope	Low	Medium	Use minimum scope (read-only events). Cache aggressively. Apply for verification only if shipping publicly.
TickTick CSV import format quirks	Medium	Low	Test on your real export early; parse leniently with clear error UI.
Apple rejection on share-extension or Siri	Low	Medium	Stay within stock App Intents and CKShare patterns. No private APIs.
Claude API costs at scale	Low (personal)	Low	Cap monthly usage per device; show estimated cost in Settings. BYOK option for power users.
Burnout on solo build	Medium	High	Strictly time-box phases. Ship MVP before adding v2 features. Use TestFlight feedback loop.

13. Cost Estimates

MVP (one-time)

- Apple Developer Program: $99/year (required for TestFlight and App Store).
- Domain name (optional): ~$12/year if you want cadence.app or similar.
- Design polish (optional): can buy SF Symbols Pro alternative icon set ~$50, or design custom in Figma free.
- Hardware: assumes you already have a Mac. If not — Mac mini M2 is the cheapest path (~$599).

Recurring (per month, personal use)

- iCloud / CloudKit: free at personal scale.
- Google Calendar API: free.
- Claude API at typical personal use (v2): ~$1.50 – $5/mo.
- TelemetryDeck (analytics): free for <10k events/day.
- Total recurring: ~$10/month, mostly the developer-program amortization + AI.

If you ship publicly

- Add small recurring cost for app analytics/CDN (~$20/mo).
- Claude API costs scale with users — solve with Cadence Plus subscription pricing as listed in 10.5.

14. Next Actions

Concrete things to do before Phase 0 begins.

- **1. Confirm co-owner's device.** iPhone vs Android changes whether v1.1 is Mac or web.
- **2. Pick the final name.** Trademark + domain check. Cadence, Tide, Loop, Daycard are top candidates from the spec.
- **3. Apple Developer Program.** Enroll if you haven't; $99/year. Required for TestFlight access.
- **4. Decide distribution model.** Personal-use-only (TestFlight forever) vs eventual App Store launch. Affects whether we worry about trademark, legal entity, and pricing.
- **5. Google Cloud project + OAuth credentials.** Free. Needed for Calendar integration in Phase 3.
- **6. Anthropic API key (for v2).** Get a key now so we can test Plan-my-day prompts during Phase 1 prototyping.
- **7. Mock review session.** Walk through the visual mockups (Cadence_Mockups.html) with your co-owner. Capture feedback before code.
- **8. Kick off Phase 0.** Create the Xcode project, push to GitHub, send your first TestFlight build — even an empty one — so the pipeline is proven before real building starts.

Appendix — Glossary of Apple terms

- **SwiftUI.** Apple's declarative UI framework, the modern way to build for iOS and Mac.
- **SwiftData.** Apple's persistence framework (replacement for Core Data), uses Codable types with @Model macro.
- **CloudKit.** Apple's iCloud-based data sync service; free at our scale.
- **CKShare.** CloudKit's mechanism for sharing data between iCloud accounts. Underpins shared lists.
- **App Group.** A shared container between the main app and its extensions (widget, Siri); lets them read/write the same local data.
- **App Intents.** Modern Siri Shortcuts framework. Replaces the old INIntent / intentdefinition file model.
- **WidgetKit.** Framework for building Home Screen and Lock Screen widgets.
- **TestFlight.** Apple's beta distribution service; install a build on a real device without going through the App Store.
- **Xcode Cloud.** Apple's built-in CI for Xcode projects; free for personal use.